



高级语言程序设计

C++

期末复习

1

题型

2

复习知识点



考试题型

- ❖ 单项选择 (20分)
- ❖ 程序改错 (16分)
- ❖ 读程序写结果 (24分)
- ❖ 程序填空 (20分)
- ❖ 程序设计 (20分)



范围

- ❖ 类和对象、静态成员与常量成员、运算符重载
- ❖ 基类和派生类、虚拟继承、多态性与虚函数、纯虚函数与抽象基类
- ❖ 函数模板、类模板、链表、栈、队列、STL的基本概念和编程方法
- ❖ 输入输出流的基本概念、格式控制、文本文件和二进制文件的输入输出

期末复习



题型



复习知识点

类和对象复习要点

❖ 类的定义

- ❑ 类名为标识符，类由成员变量和成员函数组成，注意类定义后面要加“;”，成员函数定义写在类定义体之外时，要加类名限定::

❖ 类成员的访问

- ❑ 三种访问权限，默认为private，类内是指类定义体和类的成员函数体。
- ❑ 非友元、非继承的情况下，类外无法访问类的私有成员和保护成员
- ❑ 友元可以访问任意成员，派生类可访问基类保护成员

类和对象复习要点

❖ 类的构造函数与析构函数

- ❑ 掌握构造函数与析构函数的调用顺序
- ❑ 掌握构造函数的作用和写法，特别是采用初始化符表的写法
- ❑ 拷贝构造函数（参数，调用时机）
- ❑ 委托构造函数（初始化符表）

cout <<
(j/10)

❖ 类对象的初始化

- ❑ 说明类对象的同时对其进行初始化，根据对象后面是否有参数表决定采用有参构造函数还是无参构造函数，注意对象初始化语句与构造函数的一致性。

❖ 类的静态成员

❑ 含义

❑ 说明及使用方式

❖ 类的友元

❑ 友元函数

- 在友元函数中，可通过类对象实现对类私有成员的访问
- 类对象通常是友元函数的参数

运算符重载复习要点

❖ 运算符重载函数的调用方式

❑ $a+b \leftrightarrow a.operator+(b)$, 或者 $a+b \leftrightarrow operator+(a,b)$

❖ 友元方式编写类的运算符重载函数

❑ 在类定义中增加友元函数说明, 该函数的参数全部为运算的分量

❖ 成员函数方式编写类的运算符重载函数

❑ 将重载函数说明为类的成员函数, 调用重载函数的对象为第一运算分量, 参数为其余运算分量, 注意运算分量的顺序问题

运算符重载主要问题

❖ 为某个类定义运算符重载函数

❑ 按照指定的方式（友元方式或成员函数方式）
设计运算符重载函数

❑ 注意：函数名为“operator运算符”形式，参数数量与重载方式有关，友元方式比成员函数方式多一个参数。一般情况下，重载函数要有返回值，其类型与重载运算符的运算含义有关。

运算符重载主要问题

❖ 使用重载运算符

- ❑ 根据类中运算符重载函数的定义，使用重载运算符构造运算表达式
- ❑ 注意：表达式中，运算分量的顺序与运算符重载函数的参数顺序一致

❖ 赋值运算符重载的问题

- ❑ 一般情况下，自定义的类对象之间允许使用“=”直接赋值，这是“浅拷贝”。有些情况，需要深拷贝，特别是类成员有指针变量的时候，需要自定义赋值运算符重载函数

类的继承与多态性

❖ 派生类

- ❑ 说明方式

- ❑ 派生方式及含义

❖ 单继承、多重继承、多级继承

❖ 派生类的构造函数及对象初始化

- ❑ 构造函数的格式

- ❑ 派生类与基类构造函数的执行顺序

- ❑ 派生类与基类析构函数的执行顺序

- ❑ 派生类对象的初始化方法

继承与多态复习要点

❖ 派生类的成员访问权限

❑ 根据不同的继承方式确定在派生类中的访问权限。私有成员不能在派生类中访问

❖ 派生类对象的说明

❑ 说明派生类对象的同时进行初始化，初始化顺序为：基类成员、对象成员、派生类成员；析构的顺序与构造的顺序相反。

❑ 注意在派生类的构造函数中，要包括初始化基类成员和对象成员的值

继承与多态复习要点

❖ 基类指针指向派生类对象

❑ 可以将派生类对象的地址赋值给基类指针

❖ 抽象基类与纯虚函数的定义

❑ 纯虚函数的写法：virtual 返回值 函数名（参数表） = 0;

❑ 纯虚函数只能在派生类中进行实现，不能调用纯虚函数。

❑ 包含纯虚函数的基类为抽象基类，不能说明抽象基类的对象，包括对象数组。

继承与多态主要问题

❖ 基类指针访问派生类对象，调用派生类的虚函数成员，实现多态性

❑ 设置基类指针，初始化为派生类对象，然后用基类指针调用派生类的虚函数成员

❑ 注意：用基类指针访问的派生类成员函数必须为虚函数

❖ 函数模板的定义及调用方法

- ❑ 定义函数模板的格式

- ❑ 类型参数

- ❑ 调用时，以实参的类型作为类型参数的实例

❖ 类模板的定义及使用方法

- ❑ 类模板的定义方法

- ❑ 类模板的类型参数和普通参数

- ❑ 类模板的成员函数定义方法

- ❑ 类模板的实例化方法

模板复习要点

❖ 函数模板的定义和调用

- ❑ 函数模板有类型参数和普通参数，在调用函数模板时，首先对类型参数进行实例化，实例化依据为类型参数对应实参的类型，实例化后的函数模板是一个具体的函数。
- ❑ 模板函数调用时，不进行实参到形参类型的自动转换

模板复习要点

❖ 类模板的实例化

❓ 类模板只有经过实例化才能成为一个具体类，因此，使用类模板首先要进行实例化，方法为：类模板名<类型实参,普通实参>，其中普通实参为常量。实例化后的类模板成为具体类，说明对象时，以“类模板名<类型实参,普通实参>”作为类名

❖ 链表类模板

- 链表节点类模板

```
template <typename T>
class Node
{
public:
    T num;
    Node<T>* next;

    Node (T n)
    {
        num=n;
        next=NULL;
    }
    ~Node ();
};
```

❖ 链表类模板

- 链表类模板

```
template <typename T>
class List
{
private:
    Node<T>* head;
    Node<T>* tail;
    int nodeCount; // 链表节点的数量
public:
    List();
    void Insert(T n); // 插入节点
    void Remove(T n); // 删除节点
    void Find(T n); // 查找节点
    void Print(); // 打印链表的数据项
};
```

链表的操作

- 链表节点的插入的成员函数

```
template <class T>
void List<T>::Insert(T n)
{
    Node<T>* tmp=new Node<T>(n);
    if(head==NULL)
    {
        head=tail=tmp;
        nodeCount++;
    }
}
```

模板复习要点

```
else
{
    if (n<head->num) //当前数据小于链表头结点保存的数据
    {
        tmp->next=head;
        head=tmp;
        nodeCount++;
        return;
    }
    if (n>tail->num) //当前数据大于链表尾结点保存的数据
    {
        tail->next=tmp;
        tail=tmp;
        nodeCount++;
        return;
    }
}
```

模板复习要点

```
Node<T>* curr=head;
    while (curr->next!=NULL)
    {
        if ( (curr->num<=n) && (curr->next->num>n) )
        {
            tmp->next=curr->next;
            curr->next=tmp;
            nodeCount++;
            return;
        }
        curr=curr->next;
    }
}
```

容器 (container) 就是**通用的数据结构**

- 数组 (array)
- 向量 (vector)
- 链表 (list, 双向链表)
- 单向链表 (forward_list)
- 双端队列 (deque)
- 集合 (set/ multiset)
- 映射 (map/multimap)
- 无序集合 (unordered_set)
- 无序映射 (unordered_map)

容器用来装载数据对象

容器

容器类名	特性	何时使用	头文件
array 数组	在内存中占有一块连续的空间，存储一个元素序列，但是，必须指定元素的数量	需要快速查找，不在意插入/删除的速度快慢。	<array>
vector 向量	在内存中占有一块连续的空间，存储一个元素序列。可以看作一个可自动扩充的动态数组，而且提供越界检查。 可用[]运算符直接存取数据。	需要快速查找，不在意插入/删除的速度快慢。能使用数组的地方都能使用向量。	<vector>
list 链表	双向链表，每个节点包含一个元素。列表中的每个元素均有指针指向前一个元素和下一个元素。	需要快速的插入/删除，不在意查找的速度慢，就可以使用链表。	< list >
forward_list 单向链表	单向链表，每个节点包含一个元素。	同上	<forward_list>
Deque 双端队列	在内存中不占有一块连续的空间（分段连续空间），介于向量和链表之间， 更接近向量 ，适用于由 两端 存取数据。 可用[]运算符直接存取数据。	可以提供快速的元素存取。在序列中插入/删除的速度较慢。一般不需要使用双端队列，可以转而使用 vector 或 list 。	< deque >

❖ 流的概念

❖ 主要的流类

☐ ios

☐ istream

☐ ostream

☐ iostream

☐ ifstream

☐ ofstream

☐ fstream

❖ 主要流类对象

☐ cin

☐ cout

☐ cerr

☐ clog

❖ 提取运算符和插入运算符

☐ 有些情况下需重载

❖ 格式控制

❑ 格式控制符

- 直接在<<或>>表达式中使用
—cout<<setw(5)<<a;

❑ 格式控制函数

- 需要流类对象进行调用
—cout.width(5);
—cout<<a;

❖ 常用的格式控制函数或控制符

☐ 输出宽度

☐ 输出精度

☐ 填充字符

输入输出流复习要点

❖ 格式控制函数

❑ `ios::width(int)`, `ios::precision(int)`,
`ios::fill(char)`的使用方法:

❑ `cout.width(5); cout<<"abc"<<endl;`

输入输出流复习要点

❖ 格式控制符

❑ setw (int), setprecision(int), setfill(char)
的使用方法:

❑ cout<<setw(5)<<"abc"<<endl;

输入输出流复习要点

❖ 插入提取运算符的重载函数

❑ 掌握返回值类型、参数类型，具体实现根据输入输出不同的类对象灵活设计

❑ `istream& operator>>(istream& in, complex& com);`

❑ `ostream& operator<<(ostream& out, complex com);`

❖ 文本文件与二进制文件的区别

- ❑ ASCII方式

- ❑ 二进制数方式

❖ 文件的读写过程

- ❑ 打开文件

 - 打开方式

- ❑ 读写文件

- ❑ 关闭文件

❖ 文本文件的读写

☐ 插入运算符和提取运算符

☐ get函数和put函数

☐ getline函数

❖ 二进制文件的读写

☐ read函数和write函数

❖ 文件的随机访问

☐ seekp函数与seekg函数

☐ tellp函数与tellg函数



文件读写复习要点

❖ get函数与put函数

☐ 按字符读写文本文件

❖ read函数与write函数

☐ 按字节读写二进制文件

❖ getline函数

☐ 按行读取文本文件，以'\n'为行结束符

文件读写复习要点

❖ tellg与tellp函数

❑ 获取二进制文件读/写指针的位置，以字节为单位

❖ seekg与seekp函数

❑ 设置二进制文件读/写指针的位置，以字节为单位

文件读写主要问题

❖ 读写文本文件

//从文件中读字符

fin.get(ch) ;

while (!fin.eof()) {

 //使用ch

 fin.get(ch) ;

}

//向文件中写字符

fout.put(ch) ;

文件读写主要问题

❖ 读写二进制文件

```
fout.write((char*)(&ss), sizeof(ss));  
fin.read((char*)(&ss), sizeof(ss));
```

❖ 按行读取文本文件

```
char line[81];  
ifstream infile("getline_1.cpp");  
//打开文件用于读  
infile.getline(line, 80);  
while(!infile.eof()) {  
    //尚未读到文件结束则继续循环(处理)  
    cout<<line<<endl;           //显示在屏幕上  
    infile.getline(line, 80); //再读一行  
}
```

例题

❖ 1.4 关于类和对象的描述，下列说法正确的是（ ）

- A. 类的友元函数可以访问该类的私有数据成员
- B. 不同对象的数据成员共享相同的内存空间
- C. 类的公有函数成员不能访问类的私有数据成员
- D. 可以在定义类时为该类的数据成员赋初值

例题

❖ 1.5 如果某个类TestClass的定义中只有下列原型的构造函数：`TestClass:: TestClass (int,int) ;` 则下列表达式中，能够正确初始化TestClass类对象的是 ()

- A. `TestClass Obj1; Obj1.TestClass (1, 2) ;`
- B. `TestClass Obj2[10] (1, 2) ;`
- C. `TestClass *Obj3 = new TestClass (1, 2) ;`
- D. `TestClass Obj4 = TestClass (1, 2) ;`

❖ 1.6 下列函数中，不能重载的是（ ）

- A. 类的成员函数
- B. 类的非成员函数
- C. 类的构造函数
- D. 类的析构函数

❖ 1.7 下列关于继承与多态性的描述中，哪个是正确的（ ）

- A. 私有继承是将基类的成员继承为派生类的私有成员
- B. 虚基类是只用来进行派生的类
- C. 基类中的友元函数不能被派生类继承，友元类可以被继承
- D. 函数重载时，除了名称重用之外，参数表不允许同时重用

❖ 1.9 关于函数模板和类模板，下列描述正确的是（ ）

- A. 函数模板和类模板的类型参数只能是基本数据类型
- B. 类模板的静态成员在定义类模板时创建
- C. 可以直接创建类模板的对象并用构造函数进行初始化
- D. 函数模板可以重载而且不允许参数类型转换

例题

时间类（Time）定义的框架为：

```
class Time{  
    int hour;  
    int minute;  
    int second;  
public:  
    .....  
}
```

请设计如下函数并给出完整的函数定义：

- （1）以友元的方式重载“>”运算符，实现时间类对象的比较。例如，time1为12:10:20，time2为13:15:25，则time2>time1
- （2）以成员函数的方式重载“-”运算符，实现时间类对象的减法。例如：time1为12:10:20，time2为13:15:25，time2-time1值为1:5:5。
- （3）显示时间的Time类成员函数，显示方式为hour:minute:second

例题

2010年国际足联世界杯足球赛正在南非进行，请编写程序，读取文本文件MatchResult.txt中各场比赛的比分，计算球队的积分并进行排名。

- 文件MatchResult.txt ► 类Team的定义如下：
保存的内容为：



```
class Team{  
public:  
    char name[20]; //队名  
    int points;    //积分  
    Team(){        points=0;}  
};
```

例题

请设计如下函数并给出函数的完整定义：

(1) 函数OutPut_File，功能为将Team类对象数组中保存的球队队名和积分以二进制的方式依次写入文件Standing.bin。该函数原型为：

```
void OutPut_File(Team[]);
```

(2) 主函数main，主要功能如下：

从MatchResult.txt中读取比赛信息，解析比赛信息得到队名和比分

根据比分计算球队的积分，胜一场得3分，平一场得1分，负一场得0分

A photograph of a computer setup on a wooden desk. A white keyboard and a white mouse on a dark mousepad are visible. A coiled cable is also present. The background is a solid dark brown color.

祝同学们取得好成绩!